

Indian Institute of Technology Hyderabad

Department of Computer Science and Engineering

CS6903: Network Security

Programming Assignment 1

Report

Submitted By:

Ambarish Sarkar	CS25MTECH11022
Ankit Kr Sinha	CS25MTECH11023

February 14, 2026

Anti-Plagiarism Statement

We certify that this assignment/report is our own work, based on our personal study and/or research and that we have acknowledged all material and sources used in its preparation, whether books, articles, packages, datasets, reports, lecture notes, or any other document, electronic or personal communication.

We also certify that this assignment/report has not previously been submitted for assessment/project in any other course lab, except where specific permission has been granted from all course instructors involved, or at any other time in this course, and that we have not copied in part or whole or otherwise plagiarized the work of other students and/or persons.

We pledge to uphold the principles of honesty and responsibility at CSE@IITH. In addition, we understand our responsibility to report honor violations by other students if we become aware of it.

Names: Ambarish Sarkar, Ankit Kr Sinha

Date: 14th February 2026

Signature: A.S., A.K.S.

1 Task 1: Basic File Management Service

1.1 Design Overview

Task 1 implements a simple iterative TCP server capable of processing one client at a time. The communication protocol is plaintext-based, and commands are uppercase ASCII strings.

Supported commands:

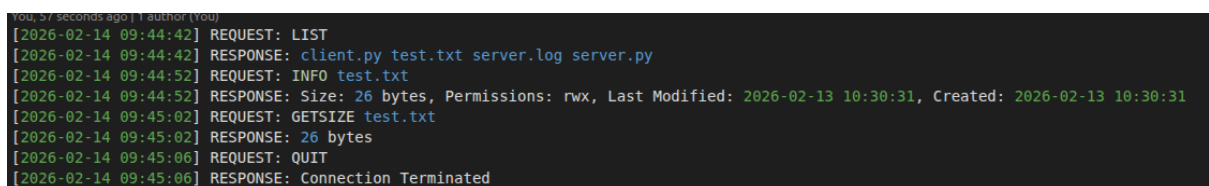
- **LIST** : Server returns the list of files in a predefined directory.
- **INFO** *< filename >* : Server returns metadata of the specified file such as last modified time, creation time, and permissions.
- **GETSIZE** *< filename >* : Server returns the size of the specified file in bytes.
- **QUIT** : Client terminates the connection.

1.2 Implementation Details

The server uses the following workflow:

- **Socket Programming**: We used the standard Python socket library. The server binds to 0.0.0.0 and enters a loop to `accept()` connections.
- **Command Parsing**: The server reads the incoming stream, decodes it from UTF-8, and splits the string to identify the command and optional arguments.
- **File Operations**: We utilized the `os` and `stat` modules to retrieve file metadata (size, permissions, timestamps) for the **INFO** command.
- **Logging**: A helper function logs every request and response to `server.log` with a timestamp, ensuring an audit trail of operations.

1.3 Sample Outputs



```
You, 57 seconds ago | 1 author (you)
[2026-02-14 09:44:42] REQUEST: LIST
[2026-02-14 09:44:42] RESPONSE: client.py test.txt server.log server.py
[2026-02-14 09:44:52] REQUEST: INFO test.txt
[2026-02-14 09:44:52] RESPONSE: Size: 26 bytes, Permissions: rwx, Last Modified: 2026-02-13 10:30:31, Created: 2026-02-13 10:30:31
[2026-02-14 09:45:02] REQUEST: GETSIZE test.txt
[2026-02-14 09:45:02] RESPONSE: 26 bytes
[2026-02-14 09:45:06] REQUEST: QUIT
[2026-02-14 09:45:06] RESPONSE: Connection Terminated
```

Figure 1.1: Server log output

```
(.venvs) ambarish-sarkar@ambarish-sarkar-L0Q-15IRX9:~/IITH/Sem 2/NS/Assignment1/NS-Assignment1/Task1$ python3 client.py
Connected to server at 127.0.0.1:8080
Available commands: LIST, INFO <file>, GETSIZE <file>, QUIT
[INPUT] LIST
[OUTPUT] RESPONSE: client.py test.txt server.log server.py
[INPUT] INFO test.txt
[OUTPUT] RESPONSE: Size: 26 bytes, Permissions: rwx, Last Modified: 2026-02-13 10:30:31, Created: 2026-02-13 10:30:31
[INPUT] GETSIZE test.txt
[OUTPUT] RESPONSE: 26 bytes
[INPUT] QUIT
```

Figure 1.2: View of Client terminal

2 Task 2: Authentication with Multiple Clients and Session Key Establishment

2.1 Design Overview

We extended the server to support **Multiple Clients** using the `threading` module, allowing simultaneous connections. We implemented a Challenge-Response authentication protocol to prevent replay attacks and used the Diffie-Hellman (DH) protocol to establish a shared session key.

2.2 Implementation Approach

- **Multi-threading:** For every new connection `accept()`, the server spawns a new thread targeting the `handle_client` function.
- **Authentication:**
 1. Server holds `username:shared_secret` in `credentials.txt`
 2. Server sends a random 64-bit nonce to the client.
 3. Client computes `SHA256(nonce + password)` and sends it back.

$$H = \text{SHA256}(N||S) \tag{1}$$

where:

- N = server-generated nonce
- S = shared secret
- 4. Server verifies the hash. If verification **fails 3 times**, the account is locked.
- **Key Exchange:** Both parties generate private keys (a and b). The client sends prime parameters (P,G) and public key A. The server computes B and the session key, then sends B back.

2.3 Account Lockout Policy

To prevent brute-force attacks, after three consecutive failed authentication attempts, the account is blocked for the session.

Since multiple clients are handled using threads, shared authentication state is protected using a mutex lock to prevent race conditions.

```

print_lock = threading.Lock()
auth_lock = threading.Lock()
.
.
.
.
< inside handleClient() >
with auth_lock:
    if username in blocked_users:
        client_socket.sendall("AUTH_FAIL: Account Blocked".encode())
        client_socket.close()
        return

```

Figure 2.1: Implementing mutex to prevent race condition

2.4 Wireshark Analysis

During Task 2, the authentication and Diffie-Hellman exchange occur in plaintext. Using Wireshark's 'Follow TCP Stream' feature, we observed:

- The username transmitted in plaintext
- The server-generated nonce
- The SHA-256 hash response
- Diffie-Hellman parameters (p, g, A)
- Server public key B

Although the password itself is never transmitted, the DH parameters are visible, confirming that encryption is not yet active in this phase.

```

(ankit-kali@ankit-kali)-[~/D/NS/Assignment-1/Task3]
$ python3 client.py
[DEBUG] Client loaded: P=23, G=5
Enter Username: ankit
Enter Shared Secret: hello
Authentication Successful!
Session Key Established: 16
-----
Commands: LIST, INFO <file>, GETSIZE <file>, GET <file>, QUIT
[INPUT] LIST
[OUTPUT] RESPONSE: test.txt client.py server.log credentials.txt server.py
[INPUT] GET test.txt
[OUTPUT] RESPONSE: FILE_FOUND 13
[INFO] Starting download with Integrity Verification...
[INFO] Download Complete. Integrity Verified.
[SUCCESS] File saved as downloaded_test.txt
[INPUT] █

```

Figure 2.2: View of client side terminal output

1. Authentication



Figure 2.3: In Wireshark, we've right-clicked a packet → Follow TCP Stream

2. Diffie-Hellman Exchange

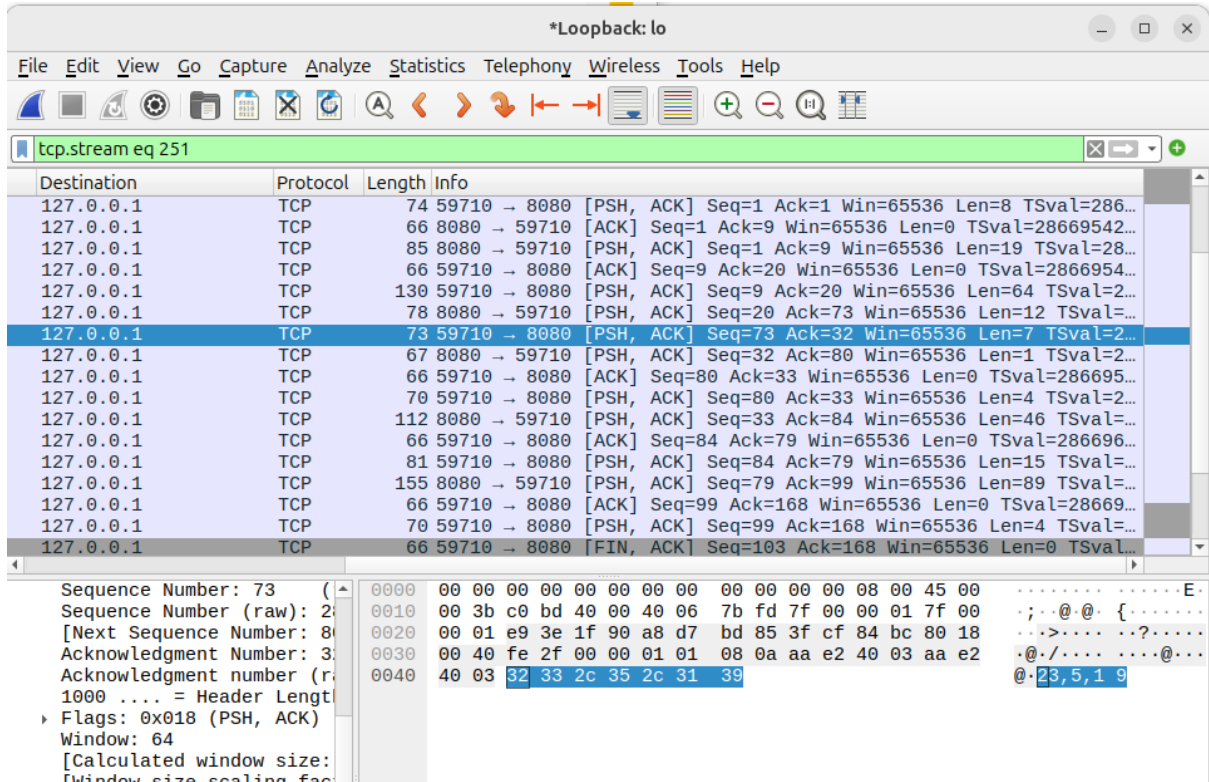


Figure 2.4: Highlighting the packet containing the "P,G,A" string (which is 23,5,19 in this case) sent by the client

Explanation: We can observe the large prime P and generator G being negotiated in cleartext, proving the DH handshake occurred.

3 Task 3: File Download with Integrity

3.1 Design

This task introduced the GET command to download files. The critical security requirement was ensuring Integrity. We designed a packet protocol where every chunk of data is accompanied by a Message Authentication Code (MAC).

3.2 Implementation

- **Packet Structure:** [SEQ_NO (4 bytes)] [DATA_LEN (4 bytes)] [DATA] [MAC (64 bytes)]
- **MAC Calculation:** We used HMAC-SHA256 logic: Hash(Data || Seq-No || Session-Key).
- **Verification:** The client recalculates the MAC for each received chunk using the established session key. If the MACs do not match, the download is aborted immediately, ensuring no corrupted data is written to disk.

3.3 Handling TCP Stream Fragmentation

Since TCP does not guarantee message boundary preservation, a helper function ensures exactly n bytes are received before processing each header and data block. This prevents packet boundary corruption.

3.4 Wireshark Analysis

In Task 3, file data remains in plaintext, but each chunk is followed by a 64-byte MAC. Using Wireshark, we observed:

- Readable file content in the TCP stream
- Sequence number and data length headers
- A 64-byte hexadecimal MAC appended after each chunk

This demonstrates that integrity protection is active, but confidentiality is not yet provided.

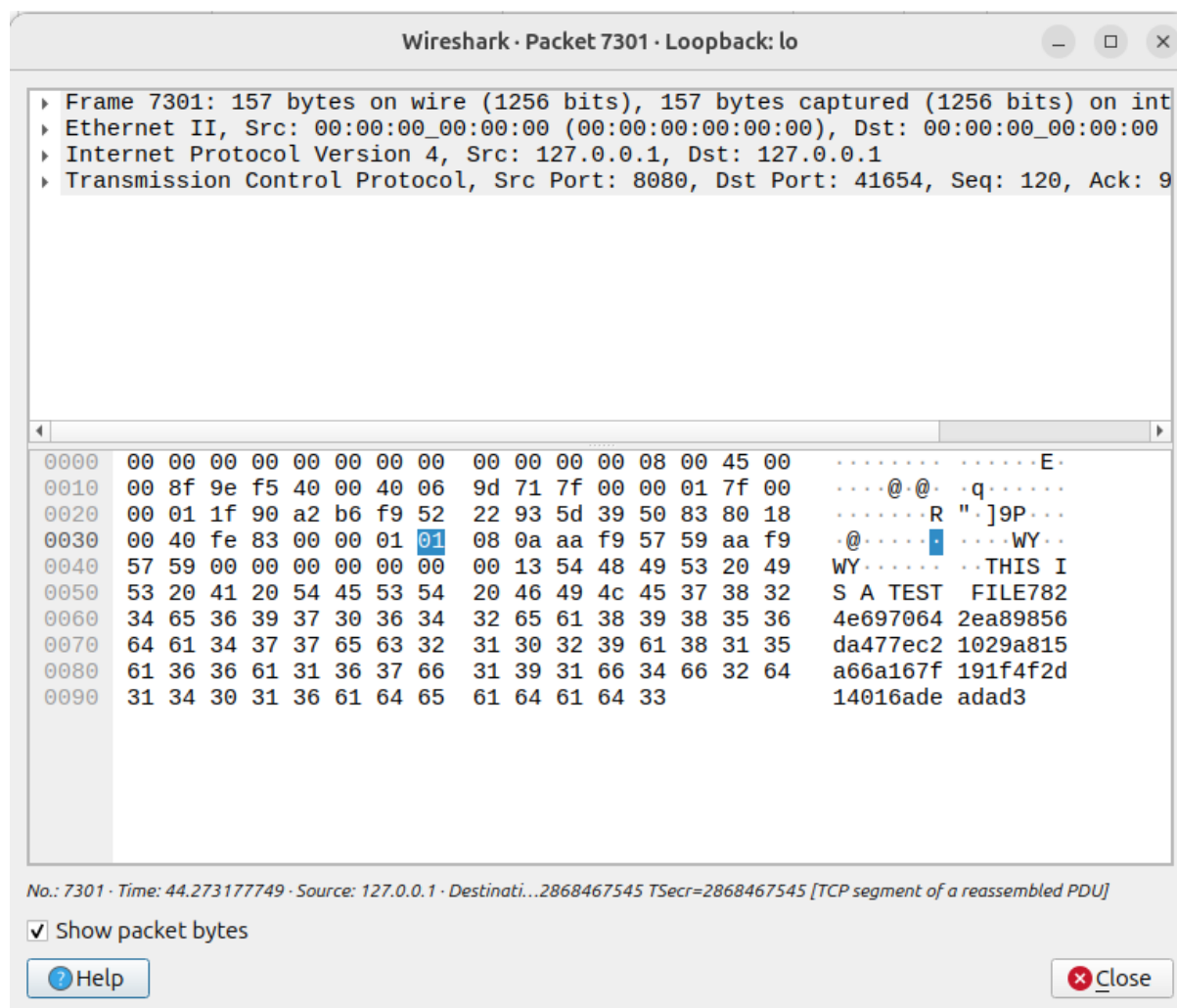


Figure 3.1: Highlighting the packet containing the contents of the test.txt file



Figure 3.2: In Wireshark, we've right-clicked a packet → Follow TCP Stream

Explanation: The capture demonstrates that while the data is visible, the 64-byte hash appended to the packet ensures that any modification in transit would be detected by the client

4 Task 4: Encryption of Communication

4.1 Design

In the final task, we secured the channel using AES Encryption to ensure Confidentiality. All communication after the initial handshake is encrypted. After successful authentication, Diffie-Hellman key exchange is performed.

4.2 Implementation Approach

- **Key Derivation:** We derived a 128-bit AES key by hashing the DH session key: `SHA256(Session_Key)[0:16]`
- **Encryption Scheme:** We used **AES-128 in CBC Mode**.
 - A random IV (Initialization Vector) is generated for every message/chunk.
 - Data is padded to 16-byte blocks (PKCS#7 padding).
 - The ciphertext sent is `IV + Encrypted_Data`.
- **Encrypted File Transfer:** The structure from Task 3 was wrapped in encryption: `SEQ_NO + Encrypt(Data) + MAC`. The MAC is calculated over the encrypted data (Encrypt-then-MAC).

4.3 Wireshark Analysis

In Task 4, after successful authentication and Diffie-Hellman exchange, all subsequent communication is encrypted using AES-128-CBC.

Observations:

- Commands are no longer readable in plaintext
- File content appears as random binary data
- Ciphertext includes a random IV for each chunk
- MAC is computed over encrypted data (Encrypt-then-MAC)

The TCP payload appears indistinguishable from random noise, confirming confidentiality.

```

(NS_venv)-(ankit-kali@ankit-kali)-[~/D/NS/Assignment-1/Task4]
$ python3 client.py
[DEBUG] Client loaded: P=23, G=5
Enter Username: ankit
Enter Shared Secret: hello
Authentication Successful!
Session Key Established. AES Key Derived.
-----
Commands: LIST, INFO <file>, GETSIZE <file>, GET <file>, QUIT
[INPUT] LIST
[Cipher]: 8f0906a34dfd25d51bc5c522c3c947b9f1f7ee74e49e4cce529539f5b919...
[Plain]: test.txt client.py server.log credentials.txt server.py downloaded_test_file.txt test_file.txt
[INPUT] GET test.txt
[Cipher]: 5dbb9540e0e5c6e8d043e0673472f2ec455ed5d88c033f7a0d938555b1f5...
[Plain]: FILE_FOUND 13
[INFO] Starting Encrypted Download...
[INFO] Download Complete. Integrity Verified.
[SUCCESS] File saved as downloaded_test.txt
[INPUT] QUIT

```

Figure 4.1: View of Client Terminal

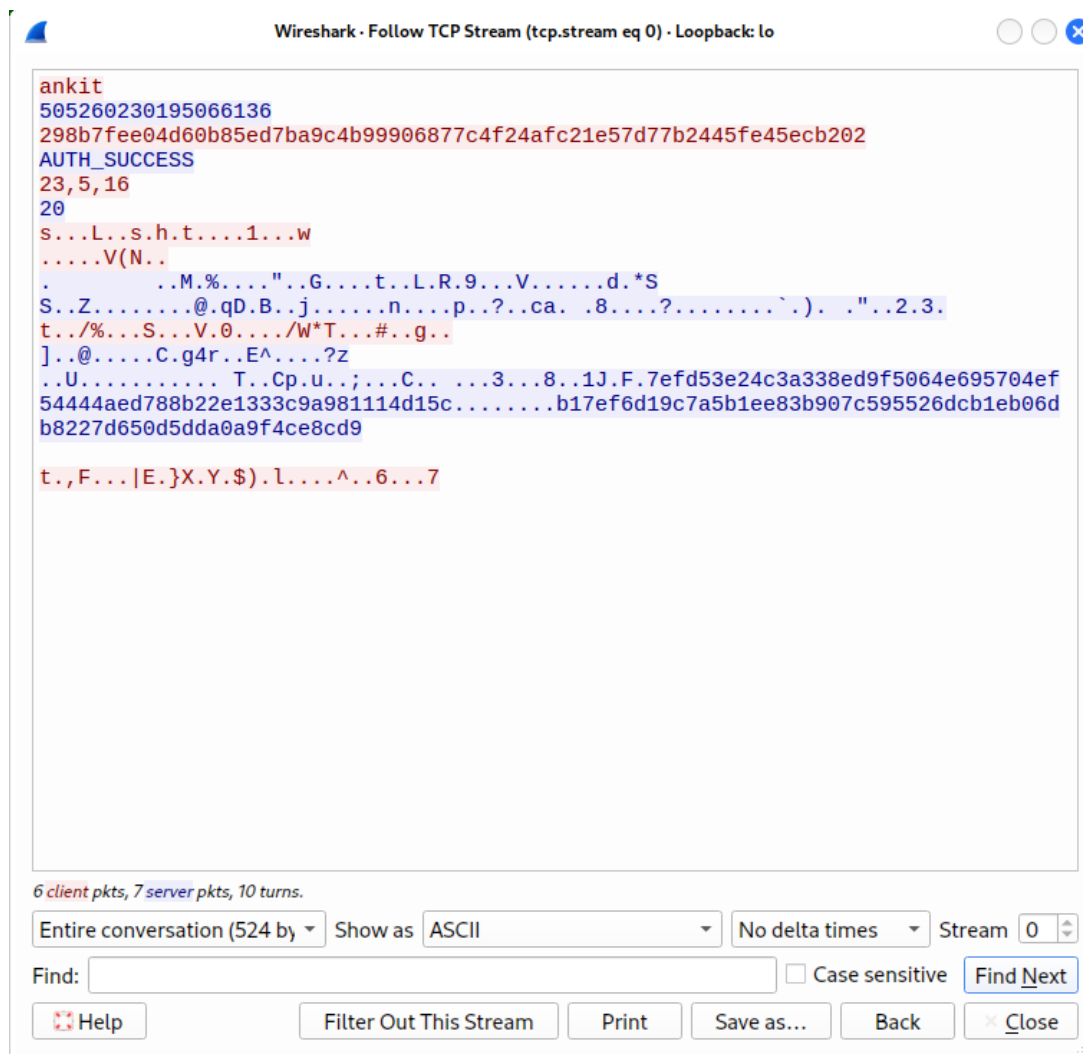


Figure 4.2: Follow TCP Stream → Highlighted that the content of test.txt file is confidential

5 Challenges Faced and Resolution

1. Thread Safety in Task 2:

- **Challenge:** When implementing the lockout policy, we realized that multiple threads accessing the `login_attempts` dictionary could cause race conditions.
- **Resolution:** We implemented a `threading.Lock()` (named `auth_lock`) to synchronize access to the global user state dictionaries.

2. Binary Data Handling in Task 3:

- **Challenge:** The `recv()` function in Python does not guarantee receiving exactly the requested number of bytes, which caused the packet parsing (Header vs. Data) to fail occasionally.
- **Resolution:** We implemented a helper function `receive_n_bytes()` that loops until exactly N bytes are received, ensuring packet boundaries are respected.

3. Environment Variable Setup:

- **Challenge:** Ensuring the server and client used the same large primes for Diffie-Hellman without hardcoding them.
- **Resolution:** We strictly followed the requirement to read P and G from `os.environ` and added debug prints to verify they were loaded correctly before the handshake.

6 Conclusion

The system effectively combines authentication, secure key exchange, integrity checks, and encryption to build a robust client-server security framework. The step-by-step improvements across tasks clearly show how fundamental cryptographic principles can be applied in real-world network security scenarios.